

SCOPE: Array-Native Nonlinear Acceleration for LLM Inference via Shape-Constrained Neural Approximation

Chenyu Zhang*, Chenxi Xu*, Jiayang Zou, Yonghao Chen, and Xinyu Chen[†]

Hong Kong University of Science and Technology (Guangzhou)

Guangzhou, China

{czhang539, cxu930, jzou521, ychen433}@connect.hkust-gz.edu.cn

xinyuchen@hkust-gz.edu.cn

Abstract—Nonlinear operators increasingly bottleneck large language model (LLM) inference as low-precision GEMMs accelerate while nonlinear functions remain on separate scalar, vector, or special-function units. Existing solutions struggle to combine broad operator support, approximation fidelity, and scalable hardware efficiency. We present SCOPE, a software-hardware co-design for array-native nonlinear evaluation. Its shape-constrained neural approximator (SCNA) encodes structural priors in a compact one-hidden-layer ReLU network. A lightweight systolic-array extension executes SCNA directly, while a scale-fused dataflow integrates it with statically quantized GEMMs. Across 11 nonlinear functions, SCNA-16 reduces geometric-mean approximation MSE by $356\times$ over reported NN-LUT values on 11 functions and $14.9\times$ over reported T-LUT values on nine functions. Across five LLMs, SCNA-8 maintains BF16/INT8 perplexity and zero-shot accuracy with limited additional degradation. In compute-focused simulations, SCOPE achieves up to $2.81\times$ INT8 long-context attention speedup over FlashAttention on TPUv6e and up to $2.69\times$ INT8 full-prefill speedup on B200 in the 512K-context extrapolation. SCNA-8 also lowers throughput-normalized incremental area and power overhead by $12.8\times$ and $9.5\times$, respectively, compared with prior nonlinear accelerators. The code is open-sourced on GitHub¹.

I. INTRODUCTION

Transformer-based large language models (LLMs) now deliver state-of-the-art results across vision and language tasks [1]–[5]. Their inference is dominated by general matrix multiplication (GEMM), so modern LLM accelerators are increasingly built around systolic arrays as the primary compute engines [6]–[8]. To further improve throughput and efficiency, modern deployment increasingly adopts low-precision formats such as INT8 for weights and activations [9]–[12]. Alongside GEMM, LLM inference also relies on nonlinear layers such as Softmax [13], activation functions such as GeLU and SwiGLU [14], [15], and normalization layers such as LayerNorm and RMSNorm [16], [17]. Their nonlinear primitives include Exp, Sigmoid, Tanh, and Rsqrt.

These nonlinear operators are emerging as a major bottleneck in modern LLM inference. Unlike GEMM, they do not

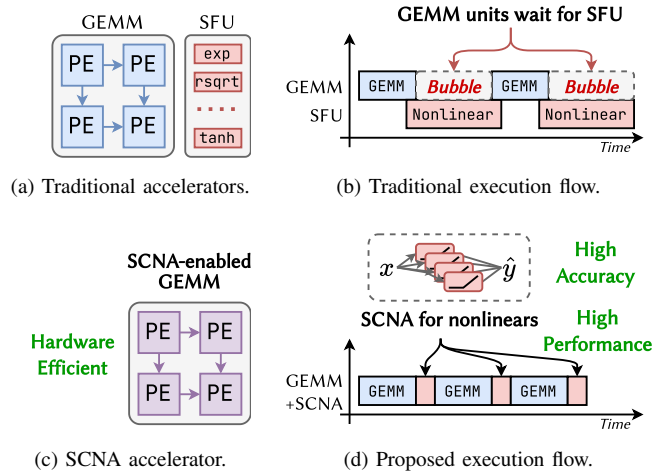


Fig. 1. SCOPE replaces separate, throughput-limited nonlinear units with shape-constrained neural approximators (SCNA) executed directly on the systolic array, making nonlinear throughput scale with GEMM resources.

map naturally to GEMM-centric hardware and are therefore typically executed on separate scalar, vector, or special function unit (SFU)-style pipelines, as illustrated in Figure 1a. Because nonlinear functions are more complex and less regular than GEMM, SFU-based implementations incur nontrivial area and power overhead and do not scale with the main systolic array [18]–[20]. As GEMM execution continues to accelerate, especially under quantization, nonlinear processing accounts for an increasingly visible share of end-to-end execution cost and becomes a system-level bottleneck, as shown in Figure 1b. Recent systems trends support this view: FlashAttention-4 redirects exponential work toward FMA resources to reduce MUFU/SFU pressure, while Blackwell Ultra/B300 increases SFU throughput to shrink softmax stalls [18], [21].

Prior work reduces nonlinear cost from several directions, but a key gap remains. Lookup-table (LUT), piecewise-linear (PWL), and learned-surrogate approaches can accelerate target functions, but high accuracy often comes with growing

*These authors contributed equally. [†]Corresponding author.

¹<https://github.com/CLab-HKUST-GZ/SCOPE-MICRO26-AE>

table capacity, finer segmentation, or more complex function-specific control [22]–[26]. Programmable nonlinear engines broaden operator coverage, yet they still treat nonlinear computation as a separate execution domain with its own mapping and control overhead [19], [20]. Array-fused designs move nonlinear processing closer to the main compute substrate, but many remain specialized or still rely on substantial auxiliary support such as LUTs, buffering, routing, or mixed-precision handling [27]–[29]. Consequently, existing approaches still fall short of making nonlinear execution broad, accurate, and natively scalable with the main array substrate.

We take a different approach. Instead of treating nonlinear functions as exceptional operations handled by separate hardware, we turn nonlinear evaluation into a capability of the systolic array itself. Our key observation is that many nonlinear operators in LLM inference exhibit strong shared structure over their practical input domains, and can therefore be represented by a compact approximation form built from affine transforms and ReLUs. This form is especially attractive for array-based execution: affine transforms are natively supported by GEMM-style systolic computation, and ReLU requires only lightweight elementwise support. As shown in Figure 1c–1d, this allows nonlinear functions to be executed on the same scalable compute substrate as GEMMs, making nonlinear execution part of the main array datapath.

We propose **SCOPE**, with the following contributions:

- We propose **SCNA**, a compact and unified nonlinear approximation method that encodes shared structural priors of nonlinear operators into a one-hidden-layer ReLU network. It improves accuracy and robustness with only a small number of neurons.
- We introduce an **array-native nonlinear execution** mechanism that maps SCNA directly onto a lightly extended systolic array, enabling scalable nonlinear execution with GEMM units.
- We present a **scale-fused quantization dataflow** that absorbs fixed input scales into offline SCNA parameterization, keeping the nonlinear path aligned with the same low-precision GEMMs.
- Our experiments show that SCNA-16 reduces geometric-mean approximation MSE by $356\times$ over reported NN-LUT values across 11 nonlinear functions. With eight neurons, **SCOPE** achieves up to $2.81\times$ INT8 attention-kernel speedup over FlashAttention in TPUv6e-based architectural simulations.

II. BACKGROUND AND MOTIVATION

A. LLM Inference and Quantization

Large language models (LLMs) are built from repeated transformer blocks [30] that combine dense matrix multiplications, namely general matrix multiplications (GEMMs), with nonlinear layers, as illustrated in Figure 2. Within each block, self-attention projects the input into query, key, and value tensors (Q , K , and V), while the feed-forward network applies two linear transformations, or three in gated variants such as SwiGLU. These stages are dominated by GEMMs.

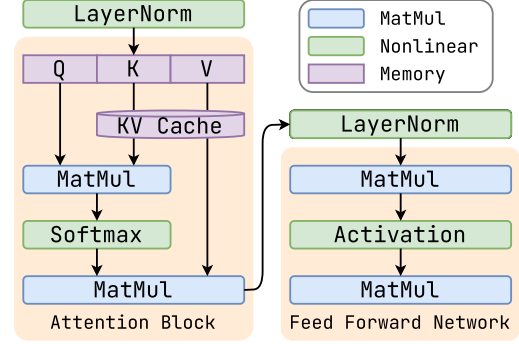


Fig. 2. Structure of a transformer layer, highlighting GEMM operations and nonlinear components in attention and feed-forward blocks.

The main nonlinear layers include `Softmax` [13] in attention [31]. The feed-forward network uses `GeLU` [14] or `SwiGLU` [15], and normalization uses `LayerNorm` [16] or `RMSNorm` [17]. These layers invoke nonlinear operators such as `Exp`, `Erf`, `Sigmoid`, and `Rsqrt`, as well as reductions and elementwise multiplication. They therefore do not naturally map to GEMM-centric hardware. Scaled dot-product attention [30] provides a representative example:

$$O = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V, \quad \text{softmax}(S)_{ij} = \frac{\exp(S_{ij})}{\sum_k \exp(S_{ik})}.$$

Here, $QK^\top$ and the final multiplication with V map naturally to GEMM accelerators, whereas `Softmax` is less compatible with systolic execution [6].

To reduce model size and memory traffic, modern LLM inference increasingly adopts low-precision formats such as INT8 or FP8 for weights and activations [9]–[12]. In quantized inference, a real-valued tensor X is typically represented by a low-precision tensor $\hat{X}$ together with a scale factor s_X :

$$\hat{X} = Q_B(X; s_X), \quad X \approx s_X \hat{X},$$

where $Q_B(\cdot)$ denotes quantization to a target format B . This representation strongly benefits GEMMs because modern accelerators typically provide much higher GEMM throughput at low precision. Nonlinear operators, in contrast, are often more numerically sensitive and therefore frequently remain in higher precision. Practical quantized LLM inference therefore commonly involves mixed-precision execution across linear and nonlinear stages, together with explicit scale handling around nonlinear layers [32], [33].

B. Nonlinear Execution Bottlenecks

The growing cost of nonlinear operators stems from execution asymmetry in modern LLM accelerators. GEMMs map efficiently to systolic-array-style tensor engines with regular dataflow and scalable throughput, and they benefit further from low-precision formats such as INT8 and FP8. Nonlinear operators, in contrast, are typically executed on narrower scalar, vector, or special function unit (SFU) pipelines, whose

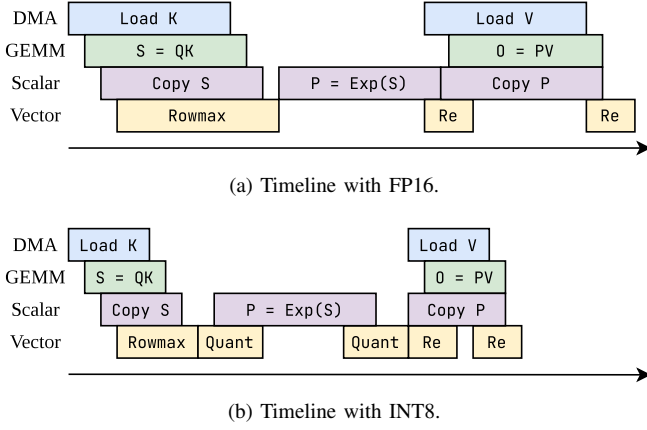


Fig. 3. FlashAttention execution timelines on AWS NeuronCore-v2 under (a) FP16 and (b) INT8. Quantization accelerates GEMMs, increasing the fraction of execution time spent on nonlinear processing.

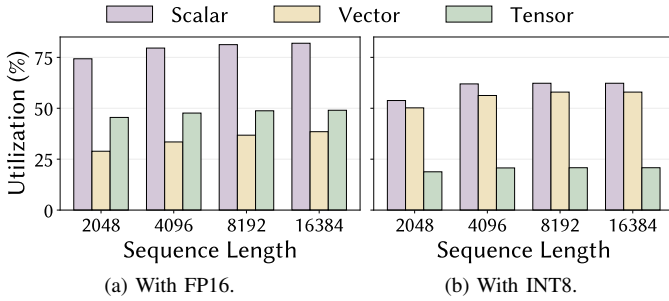


Fig. 4. Engine utilization during FlashAttention execution on AWS NeuronCore-v2. Tensor-engine utilization falls from below 50% in FP16 to around 20% in INT8, while scalar/vector utilization remains high.

throughput scales much less effectively with the main GEMM substrate because they require function-specific logic and less regular execution [7], [8].

We use FlashAttention [34], a widely used I/O-aware exact attention algorithm, as a representative example. The AWS Neuron Kernel Interface (NKI) `attention_cte` kernel exposes a similar optimized execution flow for long-sequence attention [35]. Figures 3 and 4 summarize the timeline and utilization breakdown on AWS NeuronCore-v2. In this kernel, score computation ($S = QK^T$) and output multiplication ($O = PV$) are mapped to the tensor engine, whereas the exponential stage inside `Softmax` ($P = \exp(S)$) is executed on the narrower SFU-backed scalar path. The critical path is therefore determined not only by matrix multiplication, but also by nonlinear processing. This imbalance is also reflected in Figure 4a: even under FP16, tensor-engine utilization remains below 50%, while the SFU or scalar path shows much higher activity because of the exponential stage.

The imbalance becomes even more severe under quantization. Although GEMMs become faster in INT8, nonlinear operators often remain in higher precision, introducing additional dequantization, quantization, and scaling around nonlinear stages. In the INT8 timeline of Figure 3b, these mixed-precision boundaries add extra work to the non-GEMM path. Figure 4b correspondingly shows that tensor-engine utilization

drops to around 20%, while vector activity rises substantially because of frequent format conversion and quantization. Quantization therefore accelerates the linear path more than the nonlinear path, further widening the execution imbalance.

C. Prior Nonlinear Acceleration Solutions

Prior proposals use approximation engines, programmable substrates, or array fusion. The key issue is throughput matching, not the absence of pipelining. A dedicated nonlinear pipeline can hide latency when its consumption rate matches the tensor engine’s production rate, but stalls reappear once GEMM throughput grows faster than SFU/vector throughput. This pressure is especially visible in low-precision inference, where INT8 or FP8 GEMM throughput increases while transcendental and scale-handling paths often remain narrower.

Lookup- and PWL-based approximation. One line of work replaces function-specific nonlinear units with more general approximation engines based on lookup tables or piecewise-linear (PWL) approximation [22]–[26]. T-LUT [22] reduces storage redundancy through a twofold lookup-table structure, while NN-LUT [23] further reduces the effective table size. PWL-based designs, however, still rely on breakpoint management, segment selection, and comparison logic. As approximation accuracy increases, table size, control complexity, and hardware overhead also grow.

Programmable nonlinear engines. Another direction improves flexibility by introducing programmable nonlinear substrates. PICACHU [20] proposes a plug-in coarse-grained reconfigurable array (CGRA) for LLM nonlinear operators, while Tandem [19] provides an on-chip companion processor for non-GEMM layers. These designs broaden operator coverage, but they do so through a separate nonlinear execution path with its own mapping flow and software stack. They retain the nonlinear/GEMM scaling mismatch.

Array-fused nonlinear execution. A third direction attempts to fuse nonlinear execution into the systolic array itself. SystolicAttention [27], FuseMax [28], and OneSA [29] all move part of nonlinear execution into the array, reducing reliance on separate side pipelines. This direction is promising, but existing designs still inherit one of two limitations: they either target a narrow operator family, especially `Exp` and `Softmax`, or require nontrivial auxiliary support such as lookup structures, buffering, or routing logic.

Common limitations under modern LLM inference. Besides limited operator coverage and hardware scalability, few designs fully address low-precision execution; many fall back to FP16/FP32 with extra format-conversion overhead. As a result, prior solutions do not yet provide a unified nonlinear substrate that is general, hardware-efficient, array-scalable, and compatible with modern low-precision LLM inference.

III. SCOPE OVERVIEW

A. Opportunity for Neural Approximation

We observe that neural approximation offers a promising path to unified nonlinear acceleration [36]–[39]. A shared neural approximator represents diverse operators using simple

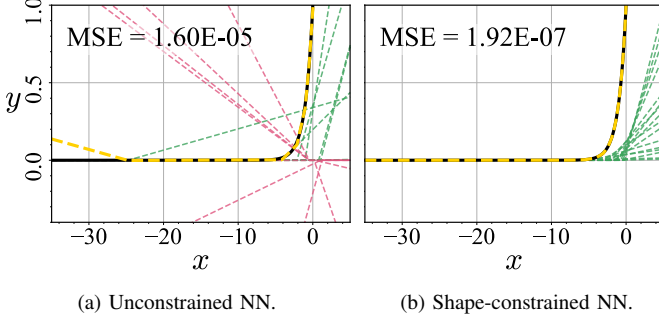


Fig. 5. Effect of shape constraints on Exp approximation. Unconstrained training mixes aligned (green) and misaligned (red) basis functions, while SCNA enforces shape consistency and reduces approximation error.

arithmetic primitives on a unified substrate. This is particularly attractive for LLM inference, where many target operators, including Softmax-related exponentials, activation functions, and normalization primitives, are structurally similar over bounded input domains. Our key insight is that these shared structural properties make them amenable to a common neural approximation form whose dominant computation aligns well with GEMM-style execution, consisting primarily of multiply-accumulate operations, bias addition, and simple activation.

B. Design Challenges

We identify three challenges:

Challenge 1: Structured rather than unconstrained approximation. Many LLM nonlinear operators share strong shape priors over their practical domains. Unconstrained neural approximation, however, fails to exploit this structure and often introduces redundant or misaligned basis components that waste model capacity and hurt robustness. The challenge is to derive a compact unified approximation form that explicitly encodes these shared priors.

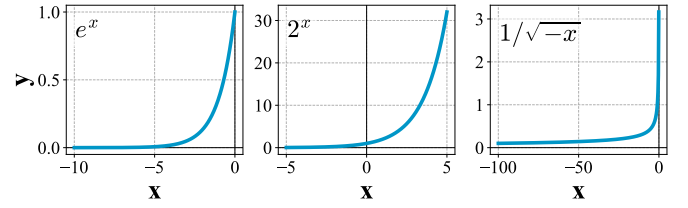
Challenge 2: Scalable execution on array-based accelerators. Even a compact approximation form is of limited value if it still requires irregular control, lookup-heavy support, or a separate nonlinear engine. The challenge is to execute nonlinear approximation on the same systolic substrate as GEMMs with regular execution, scalability, and low overhead.

Challenge 3: End-to-end compatibility with low-precision inference. Quantized LLM inference accelerates the linear path, but nonlinear stages often remain in higher precision and introduce repeated format conversion and scale-handling overhead. The challenge is to integrate nonlinear execution into a quantization-friendly pipeline so that its benefit is preserved at system level.

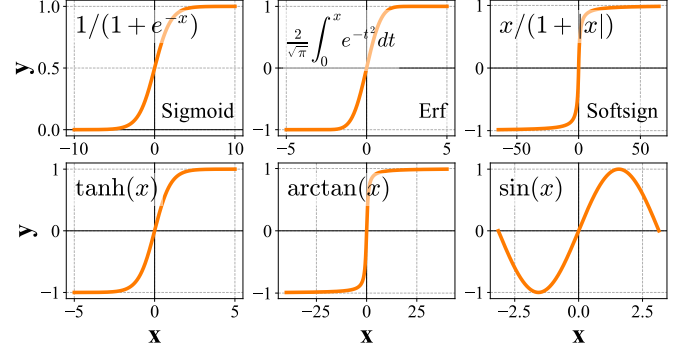
C. Technique Overview

SCOPE addresses these challenges as follows.

Technique 1: Shape-constrained neural approximation. To support diverse nonlinear operators under a tight hardware budget, SCOPE proposes a shape-constrained neural approximator that embeds structural priors such as monotonicity, symmetry, and convexity into the approximation form. This



(a) Monotone-convex primitives.



(b) Symmetric and transformed variants.

Fig. 6. Structural families of nonlinear functions in LLMs.

enables compact, unified approximation of attention, activation, and normalization operators.

Technique 2: Array-native nonlinear execution. To execute this approximation efficiently on array-based accelerators, SCOPE maps the neural approximation directly onto the systolic array with minimal architectural extension. By reusing simple arithmetic primitives that naturally fit the array datapath, it avoids LUT-heavy support and separate nonlinear side pipelines while scaling with GEMM.

Technique 3: Scale-fused quantization dataflow. SCOPE integrates nonlinear execution into the quantized inference pipeline through a scale-fused low-precision dataflow for static quantization. This dataflow reduces floating-point dequantization and requantization across operator boundaries, reduces scale-handling overhead, and helps nonlinear acceleration translate into end-to-end gains.

IV. SHAPE-CONSTRAINED NEURAL APPROXIMATORS

A. Pitfalls of Unconstrained Approximation

Several prior works use single-hidden-layer ReLU networks to approximate nonlinear functions [23], [40]. Our analysis reveals an overlooked limitation of optimizing only output error: the internal structure of the learned approximation remains unconstrained. Even when the fitted output matches the target well within the training range, the approximation can still be composed of aligned and misaligned basis functions that partially cancel each other.

Figure 5 illustrates this phenomenon for Exp. Since Exp is monotone increasing and convex, its approximation should ideally be constructed from neurons with the same trend. Unconstrained training, however, can produce a mixture of aligned basis functions (green curves) and misaligned basis

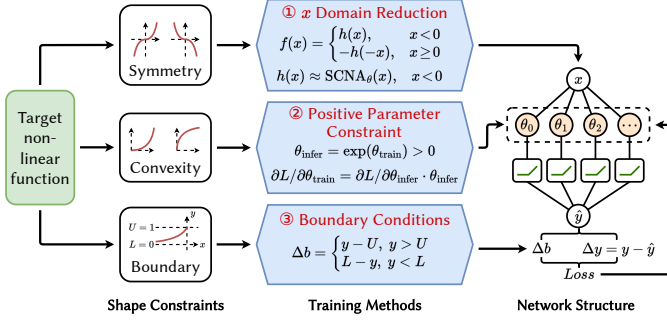


Fig. 7. SCNA domain reduction, constraints, and boundary-aware training.

functions (red curves), whose cancellation is hidden in the final output. This hidden cancellation wastes model capacity, weakens extrapolation beyond the training range, and reduces robustness under low-bit quantization.

This observation motivates *Shape-Constrained Neural Approximators* (SCNA), which enforce shape consistency during training to improve accuracy and robustness (Figure 5b).

B. Shared Shape Priors in Nonlinear Functions

Although nonlinear operators in LLM inference appear diverse, many of them can be reduced to a small set of geometric templates. Figure 6 summarizes the main structural families used by SCNA. One important template is the *monotone-convex* shape. Exp, Exp2, and reflected Rsqrt exhibit this shape on the negative half-axis.

A function f is *monotone-convex* on $x \in (-\infty, 0)$ if

Monotone: $x \leq y \Rightarrow f(x) \leq f(y)$,

Convex: $f(\alpha x_1 + (1 - \alpha)x_2) \leq \alpha f(x_1) + (1 - \alpha)f(x_2)$,

where $\alpha \in [0, 1]$. Many important operators, such as Exp and Exp2, directly satisfy this property. Several other functions do not globally follow the monotone-convex template, but they can still be reduced to it through simple transformations. For example, Rsqrt, defined as $x^{-1/2}$, is convex and monotone decreasing on $x > 0$, and can be converted into the same canonical form by input reflection. Similarly, Tanh and Sigmoid exhibit central symmetry,

$$f(x) + f(-x) = a,$$

where $a = 0$ for Tanh and $a = 1$ for Sigmoid. Moreover, both functions are monotone increasing and convex on the negative half-axis. It is therefore sufficient to approximate only the branch $x \leq 0$ and recover the positive branch by symmetry. Periodic functions such as Sin can likewise be reduced to a canonical interval $(-\pi/2, 0)$ using periodicity and reflection, where Sin is also monotone-convex. For functions with negative values, we further apply a constant shift so that the approximated canonical branch remains nonnegative.

Overall, many nonlinear operators in LLM inference can be reduced to a small number of canonical branches, among which the monotone-convex family is the dominant one.

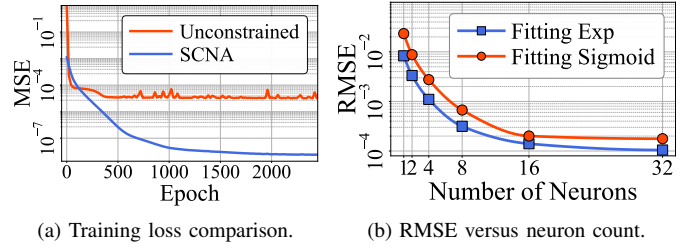


Fig. 8. SCNA training convergence and neuron efficiency.

TABLE I
REPRESENTATIVE SCNA TRAINING RECIPES.

Function	Property	Input Range	λ_{bound}
Exp	Monotone-convex	$(-256, 0)$	0.15
Exp2	Monotone-convex	$(-256, 0)$	0.15
Sigmoid	Symmetric at $(0, 0.5)$	$(-6.234, 6.234)$	0.10
Erf	Symmetric at $(0, 0)$	$(-2.469, 2.469)$	0.15
Rsqrt	Reflected at $x = 0$	$(0.1, 1024)$	0.10
Sin	Symmetric at $(0, 0)$	$(-\pi/2, \pi/2)$	0.30
Tanh	Symmetric at $(0, 0)$	$(-3.465, 3.465)$	0.20
Softsign	Symmetric at $(0, 0)$	$(-128, 128)$	0.15
Arctan	Symmetric at $(0, 0)$	$(-128, 128)$	0.15

This shared shape prior provides the foundation for *Shape-Constrained Neural Approximators* (SCNA) and enables more compact and robust approximation than unconstrained neural forms. The shape prior is a training strategy rather than a hardware restriction. The array datapath in Section V can execute any sum of affine-plus-ReLU terms, while the monotonicity, convexity, symmetry, reflection, and bound constraints make the representation compact and robust for continuous scalar nonlinearities in LLM attention, activation, and normalization.

C. SCNA Formulation

This canonical *monotone-convex* structure maps naturally to a single-hidden-layer ReLU network:

$$g(x) = \sum_{i=1}^S k_i \cdot \text{ReLU}(w_i x + b_i), \quad (1)$$

where $\text{ReLU}(\cdot)$ is *monotone-convex* and efficient in hardware.

$\text{ReLU}(x \cdot w_i + b_i)$ is the composition of an affine function $x \cdot w_i + b_i$ and $\text{ReLU}(\cdot)$. A sufficient condition for this branch to be nondecreasing is $w_i \geq 0$. Nonnegative k_i likewise preserves the monotone-convex property. We therefore require $k_i, w_i \geq 0$. Moreover, for $w_i > 0$, a basis contributing over the target domain $x \in (-\infty, 0)$ must have a non-positive hinge point: $t_i = -b_i/w_i \leq 0$. Otherwise, the corresponding component contributes zero over the target range. This condition yields $b_i \geq 0$. The case $w_i = 0$ represents a nonnegative constant basis and has no finite hinge point. Parameter fusion then simplifies Equation 1:

$$\begin{aligned} g(x) &= \sum_{i=1}^S \text{ReLU}((k_i w_i) \cdot x + (k_i b_i)) \\ &= \sum_{i=1}^S \text{ReLU}(w_i^* \cdot x + b_i^*) \end{aligned}$$

with $w_i^* \equiv k_i w_i \geq 0$ and $b_i^* \equiv k_i b_i \geq 0$. On any compact interval in the canonical negative half-axis, this parameterization therefore provides a natural approximator for nonnegative monotone-convex targets. We define SCNA as a single-hidden-layer ReLU network with nonnegative parameters:

$$\text{SCNA}_\theta(x) = \sum_{i=1}^S \text{ReLU}(w_i \cdot x + b_i), \quad (2)$$

where $\theta = \{w_i, b_i\}_{i=1}^S$ and $w_i \geq 0, b_i \geq 0$ for every i .

D. SCNA Training from Shared Shape Priors

Figure 7 summarizes the three components of SCNA training. We first reduce each target operator to a canonical branch with consistent monotone-convex structure by using symmetry or reflection. We then use exponential reparameterization to enforce the nonnegativity constraints that encode the monotone-convex shape prior. Finally, a boundary-aware loss supplements mean squared error (MSE).

Canonical branch reduction. We first transform each target function into a canonical branch that matches the monotone-convex structure exploited by SCNA.

For centrally symmetric Sigmoid, Erf, Tanh, and Softsign, we learn the negative branch $h(x)$:

$$f(x) = \begin{cases} h(x), & x < 0, \\ a/2, & x = 0, \\ a - h(-x), & x > 0. \end{cases} \quad (3)$$

where $a = 1$ for Sigmoid and 0 for the other listed functions.

For a branch with negative outputs, we fit the shifted target $h(x) + c \geq 0$, then recover $\hat{h}(x) = \text{SCNA}_\theta(x) - c$. We use $c = 1$ for Tanh, Erf, Softsign, and the canonical Sin branch, and $c = \pi/2$ for Arctan. Positive inputs use $a - \hat{h}(-x)$. The exact value at zero is $a/2$; finite fitting error at the branch endpoint can still produce a small reconstruction discontinuity.

For decreasing convex functions defined on $(0, \infty)$, such as $1/x$ and $1/\sqrt{x}$, we reflect the function about $x = 0$ and train $h(x)$ on the negative half-axis: $f(x) = h(-x)$. If $f(x)$ is decreasing and convex on $(0, \infty)$, then $f(-x)$ is increasing and convex on $(-\infty, 0)$. Softplus is fitted directly on $[-16, 0]$, with positive inputs reconstructed using $\text{Softplus}(x) = x + \text{Softplus}(-x)$. For GeLU, SCNA fits $1 + \text{Erf}(x/\sqrt{2})$ on $[-8, 0]$; the full operator additionally multiplies the reconstructed branch by $x/2$. The reported Rsqrt errors cover $[0.1, 1024]$. After these transformations, SCNA trains on a canonical branch with shape priors.

Positive reparameterization. Once the target is reduced to a canonical monotone-convex branch, training must preserve the nonnegativity constraints of SCNA. One way to enforce $w_i, b_i \geq 0$ is clipping after each optimization step:

$$w_{\text{new}} \leftarrow \max(w_{\text{opt}}, \epsilon), \quad \epsilon > 0$$

However, this approach can truncate parameter updates from the optimizer and affect convergence smoothness and approximation quality [41], [42]. Instead, we use *positive reparameterization* to replace each constrained parameter by the exponential of an unconstrained raw parameter [43], [44]:

$$\begin{aligned} \text{SCNA}_\theta(x) &= \sum_{i=1}^S \text{ReLU}(w_i \cdot x + b_i) \\ &= \sum_{i=1}^S \text{ReLU}(e^{w_{i,\text{raw}}} \cdot x + e^{b_{i,\text{raw}}}) \end{aligned} \quad (4)$$

where $w_{i,\text{raw}}, b_{i,\text{raw}} \in \mathbb{R}$. This transformation guarantees non-negativity while preserving smooth gradients for optimization. We therefore train the model on the raw parameters $\theta_{\text{raw}} = \{w_{i,\text{raw}}, b_{i,\text{raw}}\}_{i=1}^S$, and deploy it with the effective positive parameters $\theta = \{w_i, b_i\}_{i=1}^S$ with $w_i = e^{w_{i,\text{raw}}}$, $b_i = e^{b_{i,\text{raw}}}$.

Boundary-aware training. Many target operators are bounded or partially bounded, and these output limits are part of their shape prior. A pure MSE objective does not encode such constraints. We therefore use

$$L_{\text{total}} = (1 - 2\lambda_{\text{bound}})L_{\text{MSE}} + \lambda_{\text{bound}}L_{\text{bound}},$$

with $0 \leq \lambda_{\text{bound}} < 1/2$. For a batch of M samples,

$$L_{\text{bound}} = \frac{1}{M} \sum_{m=1}^M [\text{ReLU}(\hat{y}_m - y_{\text{max}})^2 + \text{ReLU}(y_{\text{min}} - \hat{y}_m)^2].$$

Here, $\hat{y}_m$ is the approximation result, and $[y_{\text{min}}, y_{\text{max}}]$ is the valid output interval for the canonical branch, including any output shift used during training. The penalty encourages valid outputs but does not impose a hard bound.

Figure 8a shows that, on Exp, the constrained formulation converges faster and more smoothly than an unconstrained neural baseline while also achieving substantially lower final error. Figure 8b further highlights the neuron efficiency of SCNA by plotting final root mean squared error (RMSE) against neuron count. The RMSE drops markedly as the neuron count increases from 4 to 16, but decreases only marginally beyond 16, indicating that 16 neurons capture most of the standalone approximation benefit for the target functions. Table I lists representative training recipes.

V. ARRAY-NATIVE EXECUTION OF SCNA

A. Systolic Array Extension

SCNA consists of repeated affine transformations followed by ReLU accumulation: $y' = y + \text{ReLU}(w_i x + b_i)$. This structure matches the native computation pattern of systolic arrays, which already provide multiply-accumulate (MAC) datapaths and regular nearest-neighbor dataflow. We therefore execute SCNA directly on the array with a lightweight extension.

Architecture overview. As shown in Figure 9a, the overall organization remains identical to a conventional systolic-array accelerator: a central two-dimensional processing-element (PE) array is coupled with a scratchpad for weights and activations, and an accumulator (ACC) for near-memory accumulation and post-processing. During standard GEMM execution, weights are preloaded from the scratchpad into the array, activations are streamed through the array, and output partial sums are accumulated along the reduction dimension in the ACC. The ACC writes back or restreams into the array.

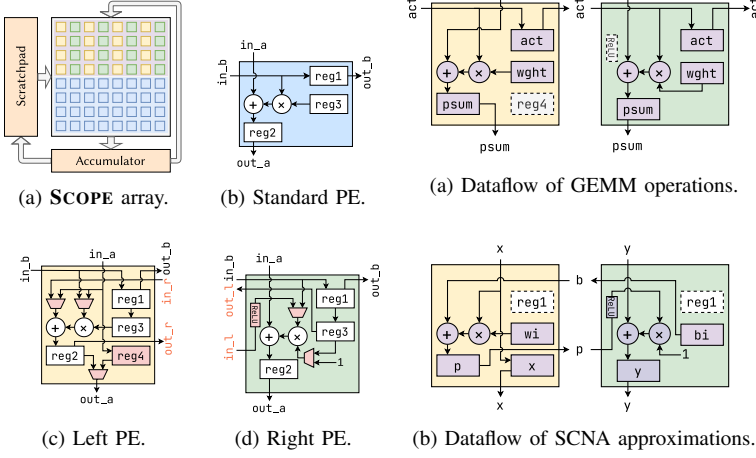


Fig. 9. SCNA array microarchitecture.

Fig. 10. Array-native SCNA dataflow.

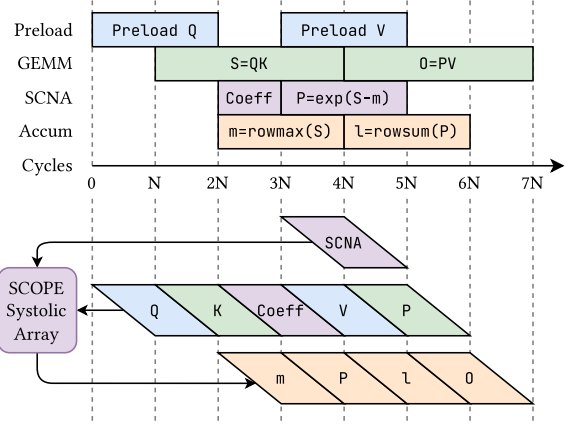


Fig. 11. Cycle-level dataflow of fused FlashAttention.

For attention, a stationary operand is a K or V activation tile rather than a persistent model-weight tile; preload overlap therefore refers to reuse within the tiled attention schedule.

Lightweight PE-pair extension. The key modification is a heterogeneous PE organization, illustrated in Figure 9a. We use three PE variants: a standard PE (blue, Figure 9b), a left PE (yellow, Figure 9c), and a right PE (green, Figure 9d). All three variants preserve the preload and GEMM semantics of a conventional systolic array with identical latency. Compared with a standard PE, the extension is deliberately lightweight: the left PE adds only one extra register reg4 with the width of one input element, the right PE adds only one small ReLU unit, and the new links are strictly local to adjacent PEs.

B. Unified Execution of GEMM and SCNA

Our systolic extension supports both standard GEMM operations and SCNA evaluation. The GEMM dataflow and latency remain identical to the base array, while the SCNA dataflow leverages the new PE-pair structure to realize high-throughput nonlinear approximation. SCNA execution can also overlap with the next GEMM tile preload because it uses the vertical stream and local PE-pair state, allowing phase switching without a separate nonlinear pipeline.

GEMM execution. Figure 10a shows the dataflow when the extended PEs execute ordinary GEMM operations. In this mode, the behavior is identical to that of a standard PE: $p'_{\text{sum}} = p_{\text{sum}} + w \times a$, where activation a moves horizontally, partial sum p_{sum} moves vertically, and weight w is preloaded locally.

SCNA execution. The key difference appears in SCNA mode, shown in Figure 10b. A left-right PE pair jointly executes one SCNA neuron. The left PE computes the affine term $p = x \cdot w_i + b_i$, where x flows vertically and is staged by reg4 , w_i is stored locally in the left PE, and b_i is stored in the adjacent right PE. The intermediate result p is then sent to the right PE, which computes $y' = y + \text{ReLU}(p)$, where y flows vertically as the running accumulated output. The addition is implemented by reusing the existing MAC datapath, with one multiplier input fixed to 1. One adjacent PE

pair therefore evaluates one SCNA neuron with one-element-per-cycle throughput: $y' = y + \text{ReLU}(w_i x + b_i)$.

Overlap with weight preload. SCNA execution uses only the vertical dataflow and a small set of registers within the PE pair, leaving the horizontal dataflow and reg1 free, as shown in Figure 10b. This permits direct overlap with weight preloading, which requires only data propagation and does not occupy MAC units. The overlap is critical for achieving consistent high throughput in kernels with rapid linear–nonlinear switching, such as FlashAttention, especially for systolic arrays with large N where preload latency is significant. The PE can copy preloaded weights from reg1 to reg3 , preserving the GEMM dataflow.

Throughput analysis. For an SCNA with S neurons, S PE pairs are vertically stacked to evaluate the full approximator. On an $N \times N$ systolic array, one vertical strip of size $N \times S$ contains $N/2$ such SCNA stacks, providing $N/2$ elements-per-cycle throughput with latency $S + 1$. Since a conventional systolic array typically accepts N elements per cycle, at most two SCNA strips, occupying $N \times 2S$ PEs in total, need to be converted, while other PEs remain unmodified.

This throughput assumes even N and $2S \leq N$, with a converted-PE fraction of $2S/N$; larger approximators need extra passes or remapping.

FlashAttention dataflow. Our FlashAttention [34] mapping is designed to overlap Exp with data movement between GEMMs, as shown in Figure 11. Once partial score outputs from the first GEMM QK^T begin to emerge at cycle $2N$, ACC starts the row-wise max reduction, and during cycles $3N \sim 5N$, as SA drains the score tile S , ACC restreams each row to SA for SCNA-based $\exp(S - \hat{m})$ after its required maximum becomes available. This Exp phase overlaps with data movement that is already required after the first GEMM and is further hidden by the preload of the next V tile for the second GEMM. Starting at $4N$, the exponentiated tile P streams out, enabling the row-wise sum reduction and the startup of PV to proceed in parallel. As a result, Exp is overlapped precisely in the drain/restream window

between the two GEMMs, removing the main serialization point of online Softmax and yielding a modeled steady-state initiation interval of $5N + \mathcal{O}(1)$ cycles.

This interval assumes matched N -sized tiles and sufficient local streaming bandwidth, and excludes pipeline fill and final drain; Figure 11 shows a window through $7N$. A PE’s MAC executes GEMM and SCNA in distinct phases; overlap uses preload, ACC reduction, and streaming resources. Online Softmax also updates the running maximum, normalizer, and output state, including rescaling when the maximum changes.

VI. SCALE-FUSED QUANTIZATION DATAFLOW

Prior low-precision nonlinear acceleration reduces the cost of nonlinear evaluation, but it still leaves an explicit boundary between quantized computation and real-valued scale semantics. For static quantization, our scale-fused design removes floating-point dequantization at this boundary by folding scale handling directly into the nonlinear datapath. Figure 12 compares the three dataflows considered in this section.

A. Mixed-Precision Boundary in Quantized LLM Inference

Figure 12a illustrates conventional quantized LLM inference. Low-bit execution is used for GEMM, but the following nonlinear stage typically restores the quantized result to full precision before evaluation [32], [33]. This dequantize–nonlinear–requantize sequence introduces extra conversion overhead and prevents nonlinear throughput from scaling with the low-bit GEMM datapath.

Figure 12b shows a stronger baseline in which the nonlinear stage also operates on low-precision data [45]. Even this baseline does not remove the core obstacle, because scale propagation through a nonlinear operator is not straightforward. For a quantized GEMM with weights $\hat{W}$, activations $\hat{A}$, and scales s_W, s_A , linearity gives

$$WA \approx (s_W s_A)(\hat{W}\hat{A}).$$

The scale therefore passes through GEMM directly, yielding output scale $s_O = s_W s_A$ and quantized output $\hat{O} = \hat{W}\hat{A}$. A nonlinear function, in contrast, does not in general admit such a direct scale mapping. Translating the input scale into the correct output scale is operator-dependent and nontrivial.

This is why a dequantization step is typically required to recover the unscaled real-valued input before applying the nonlinear function with its function-specific input/output scales and then representing the result back in the quantized numerical domain. Consequently, fusing the Dequant and Quant steps, as illustrated in Figure 12c, is nontrivial: it must correctly handle the input scale of the nonlinear function, and this handling is usually specific to each nonlinear operator.

B. Fusing Quantization Scales into SCNA

Our key observation is that ReLU, despite being nonlinear, is positively homogeneous: $\text{ReLU}(sx) = s \text{ReLU}(x)$ for $s > 0$. Because SCNA is a sum of ReLU bases, it inherits a structured scale-mapping property that lets us propagate quantization scales through the nonlinear stage much like

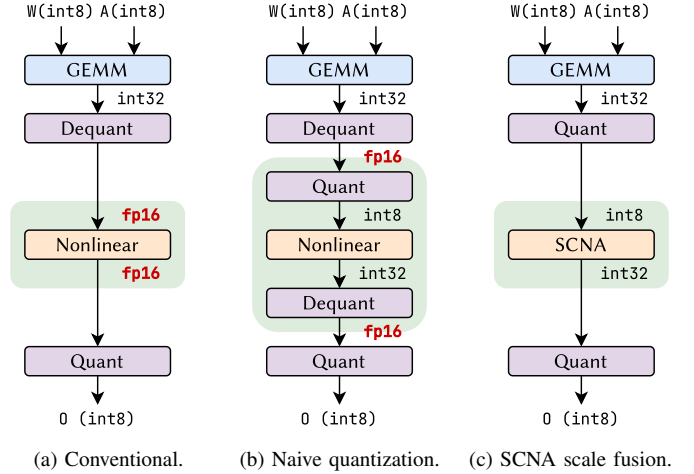


Fig. 12. Nonlinear evaluation for quantized execution.

through GEMM. This property enables the optimized dataflow in Figure 12c, where nonlinear evaluation uses low-precision inputs without floating-point dequantization.

Let $\text{SCNA}_\theta(\cdot)$, with $\theta = \{w_i, b_i\}_{i=1}^S$, denote the SCNA defined in Equation 2. For any positive scale $s > 0$,

$$\begin{aligned} \text{SCNA}_\theta(sx) &= \sum_{i=1}^S \text{ReLU}(w_i sx + b_i) \\ &= s \sum_{i=1}^S \text{ReLU}(w_i x + b_i/s) \\ &= s \cdot \text{SCNA}_{\theta_s}(x), \end{aligned} \quad (5)$$

where $\theta_s = \{w_i, b_i/s\}_{i=1}^S$. Thus, in real arithmetic, SCNA preserves the input scale at the output, provided that the scale s is absorbed into a reparameterized basis set θ_s . Under static quantization, the required input scales are fixed after calibration, so this reparameterization can be performed once offline.

For requantization, we use $Q_B(z; s) = \text{clip}_B(\text{round}(z/s))$, with clipping to the representable range of format B . This gives the scale-fused dataflow in Figure 12c. The upstream quantized GEMM produces an INT32 accumulation $\hat{G} = \hat{W}\hat{A}$ with scale $s_G = s_W s_A$, which is fixed under static calibration. We then apply one requantization $\hat{P} = Q_8(\hat{G}; s_P)$ to obtain the INT8 SCNA input $\hat{P}$, whose effective scale is $s_W s_A s_P$. Instead of dequantizing $\hat{P}$, we fuse this scale into the SCNA parameters offline by replacing θ with $\theta_{s_W s_A s_P}$, that is, by dividing each bias b_i by $s_W s_A s_P$. For an integer realization, the transformed coefficients also require quantization: with $s = s_W s_A s_P$ and a common coefficient scale $s_c > 0$, let $w_i \approx s_c \hat{w}_i$ and $b_i/s \approx s_c \hat{b}_i$. The nonlinear stage can then operate directly on $\hat{P}$, producing an INT32 output

$$\hat{Q} = \sum_{i=1}^S \text{ReLU}(\hat{w}_i \hat{P} + \hat{b}_i),$$

with effective scale $s_W s_A s_P s_c$. If this result feeds the next quantized GEMM, we apply a final static requantization $\hat{O} = Q_8(\hat{Q}; s_Q)$, yielding output $\hat{O}$ with total scale $s_W s_A s_P s_c s_Q$.

Output shifts and symmetry reconstruction must use the accumulated scale before final requantization. Parameter and input-range calibration must avoid INT32 overflow; coefficient rounding and clipping add numerical error.

The nonlinear path therefore uses INT8 inputs and outputs with INT32 accumulation. Only the accumulated real-valued scale is carried as metadata rather than materialized at the nonlinear boundary. Compared with the naive low-precision flow in Figure 12b, the key change is that the nonlinear boundary becomes a low-precision requantization rather than a floating-point dequantize–requantize boundary.

The preprocessing overhead is minimal. Static quantized deployment already calibrates layer scales and activation clipping ranges, and SCNA already performs offline fitting of its basis parameters. Scale fusion adds a layer-local reparameterization of the SCNA biases using the fixed calibrated input scale, followed by coefficient quantization.

The offline preprocessing assumes static input scales. Dynamic-quantization accuracy results in Section VII-C3 demonstrate numerical compatibility, but varying scales require coefficient updates or explicit scale conversion.

VII. EVALUATION

A. Experimental Setup

Function-level accuracy. We study a broad family of nonlinear functions covering the main nonlinear operators in LLMs. The set includes attention-path functions (Exp and Exp2), activation-path functions (Sigmoid, Tanh, Softplus, Erf, and the nonlinear branch of GeLU), the normalization primitive Rsqrt, and additional targets (Softsign, Arctan, and Sin). We use floating-point reference functions for SCNA and compare against reported values for classical approximation methods and prior PWL-based approximators. We use 16 neurons for standalone function accuracy by default unless otherwise specified. For scalability analysis, we vary the neuron count and measure how the approximation error changes with model capacity.

Model-level accuracy. We evaluate five widely used open-source LLMs: OPT-6.7B, Llama-2-7B, Llama-3-8B, Qwen2.5-7B, and Qwen3-8B. For the quantization setting, we quantize weights, activations, and the KV cache to INT8 statically with SmoothQuant [10]. For activation calibration, we use The Pile [46] with 128 sequences of 2048 tokens each. We measure end-to-end model quality using perplexity on WikiText-2 and zero-shot accuracy on ARC-Easy [47], HellaSwag [48], PIQA [49], and WinoGrande [50] with lm-eval [51].

End-to-end performance. We evaluate end-to-end performance with an extended LLMCompass [52], which uses SCALE-Sim [53] for cycle-accurate systolic-array simulation and whose original study reports 0.69% average prefill error on its validation workloads, not on our SCNA extensions or new device configurations. We model FlashAttention following FlashAttention-2 [34], integrate the SCNA-enabled array into the same simulator, and validate its timing against register-transfer-level (RTL) simulation. We use 8 neurons for performance by default to balance accuracy and efficiency. For the

TABLE II
DEVICES USED FOR END-TO-END PERFORMANCE EVALUATION.

Metric		B200 [54]	AWSv4 [55]	TPUv6e [56]
Systolic Array	Count	592	8	2
	Size	16×16	128×128	256×256
Throughput (TOPS)	FP16 GEMM	2242.970	671.089	917.504
	INT8 GEMM	4485.939 (2×)	2684.355 (4×)	1835.008 (2×)
	Vector	70.093	9.830	7.168
	SFU / Exp	4.381	39.322	3.584
FP16 GEMM / Exp ratio		512	17	256
Memory	Capacity	180 GB	144 GiB	32 GB
	Bandwidth	8.000 TB/s	4.900 TB/s	1.600 TB/s

TABLE III
PREFILL ATTENTION THROUGHPUT (TFLOP/S) COMPARISON WITH INTEGER-ONLY NONLINEAR BASELINES.

Length	2K	4K	8K	16K
SCOPE	1130.86	1526.51	1672.82	1713.89
I-LLM	641.65 (-43.3%)	746.36 (-51.1%)	772.04 (-53.8%)	782.08 (-54.4%)
IntAttention	888.73 (-21.4%)	1093.30 (-28.4%)	1170.41 (-30.0%)	1104.60 (-35.5%)

hardware baselines in Table II, we import device specifications into LLMCompass and replace only the baseline systolic array with the SCNA-enabled variant. The GPU array organization is a throughput-model abstraction. All attention sweeps disable HBM bandwidth limits. On-chip I/O limits are enabled for AWSv4 and TPUv6e, but disabled for B200.

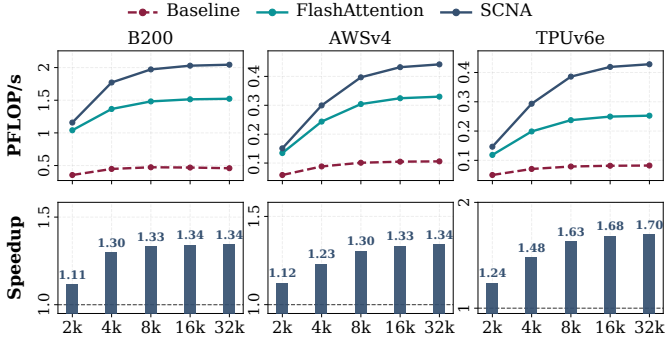
Baselines. Our baselines include the lookup- and PWL-based approximators LinearLUT [24], NN-LUT [23], and T-LUT [22], as well as array-fused nonlinear accelerators including SystolicAttention (denoted as FSA) [27], FuseMax [28], OneSA [29], as well as the standalone programmable engine PICACHU [20]. We also add a B300-style sensitivity point that doubles baseline SFU throughput following Blackwell Ultra [21] and discuss integer-only nonlinear flows represented by I-LLM [57] and IntAttention [58].

Hardware implementation. We implement the SCNA-enabled systolic array in Chisel [59] and synthesize the generated Verilog with Synopsys Design Compiler in TSMC 28 nm at 1 GHz. Floating-point units use DesignWare IP [60].

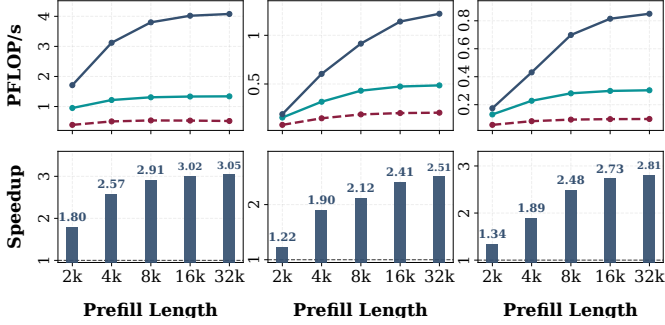
B. End-to-end LLM Performance

1) Attention-kernel Throughput: We evaluate attention-kernel prefill throughput over sequence lengths from 2K to 32K under both FP16 and INT8 execution. We report effective throughput in FLOPs/s as $4BHN^2d/T$, where $B = 1$ is batch size, $H = 32$ is the head count, N is sequence length, $d = 128$ is head dimension, and T is kernel latency. This counts QK^T and PV with two operations per multiply-accumulate, excluding projections and nonlinear work, and uses the dense-attention convention without causal triangular-work reduction. INT8 uses the same operation count. “Baseline” denotes unfused attention; “FlashAttention” denotes fused attention.

Figure 13 reports results on three representative accelerators, as listed in Table II, together with the speedup of SCNA over a FlashAttention baseline. The results show that SCNA delivers the highest throughput in every evaluated setting, and



(a) Full-precision FP16.



(b) Static INT8 quantization.

Fig. 13. Prefill attention throughput and speedup over FlashAttention.

its advantage consistently increases with context length. In FP16, SCNA achieves a $1.35\times$ geometric-mean speedup over FlashAttention, with gains reaching $1.34\times$ – $1.70\times$ at 32K.

The benefit is even larger in INT8, which directly validates our motivation. Moving from FP16 to INT8 substantially increases GEMM throughput on modern accelerators, $2\times$ in general and up to $4\times$ on AWSv4, but the throughput of conventional nonlinear units does not scale proportionally. Nonlinearity therefore becomes a primary bottleneck. SCNA addresses this mismatch by providing an array-native nonlinear execution path whose throughput scales much more closely with the surrounding attention pipeline. As a result, the maximum speedup increases from $1.34\times$ to $3.05\times$ on B200, from $1.34\times$ to $2.51\times$ on AWSv4, and from $1.70\times$ to $2.81\times$ on TPUv6e when moving from FP16 to INT8.

2) *Full-Model Prefill Throughput*: We also model full-model transformer prefill latency, including attention, FFN, normalization, residual, final-normalization, and output-projection components. The modeled decoder has 32 layers, hidden size 4096, 32 attention heads, FFN width 14,336, full-width Q/K/V projections, batch size one, and one output logit position. Non-attention operator estimates disable HBM and on-chip I/O bandwidth limits. Results through 32K reuse the attention sweep; 64K–512K results extrapolate its fixed tile mappings and do not establish checkpoint context support or memory-capacity feasibility.

Figure 14 shows that attention-kernel gains translate into end-to-end speedup and grow with context length. At 32K context, FP16 full-prefill speedup is $1.207\times$ on AWSv4, $1.183\times$

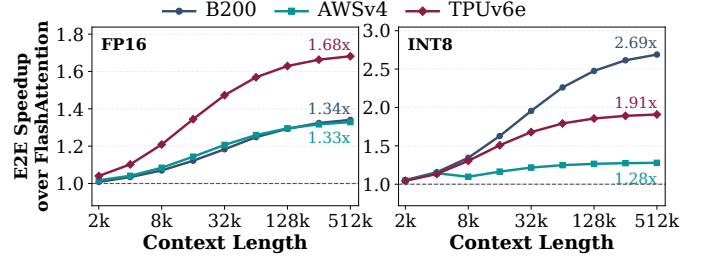


Fig. 14. Full-prefill speedup over FlashAttention at long contexts.

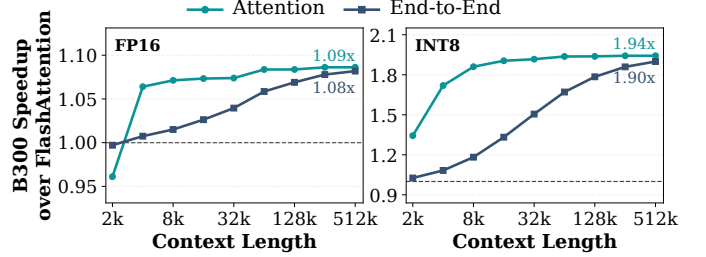


Fig. 15. B300-style sensitivity to increased SFU/MUFU throughput.

on B200, and $1.473\times$ on TPUv6e. At 512K context, the FP16 full-prefill speedup increases to $1.329\times$, $1.341\times$, and $1.681\times$, respectively; under INT8, the longest-context endpoints reach $1.28\times$, $2.69\times$, and $1.91\times$ on AWSv4, B200, and TPUv6e. This Amdahl-style trend is expected because the accelerated attention fraction grows with context length.

3) *Comparison to a B300-Style Baseline with Doubled SFU Throughput*: Figure 15 evaluates a B300/Blackwell Ultra-style baseline with doubled SFU/MUFU throughput. It reduces exposed softmax stalls, especially in FP16, but does not eliminate the mismatch. At the longest context, **SCOPE** still provides $1.09\times$ attention and $1.08\times$ full-prefill speedup in FP16, and $1.94\times$ and $1.90\times$, respectively, in INT8. Thus, higher SFU throughput helps, but array-native execution complements it as low-precision GEMMs accelerate.

4) *Comparison to Prior Fully Integer Nonlinear Flows*: Table III compares modeled H100 INT8 prefill attention throughput with the pure-integer I-LLM and IntAttention baselines. We implemented their softmax algorithms as optimized Triton kernels, benchmarked them against reference softmax on an H100, and used the measured ratios to scale the reference softmax stage in the attention model, which combines it with GEMM and pipeline costs. **SCOPE** increases from 1130.86 TFLOP/s at 2K to 1713.89 TFLOP/s at 16K. I-LLM trails **SCOPE** by 43.3% at 2K and 54.4% at 16K; IntAttention trails by 21.4%–35.5%.

C. Approximation Accuracy

1) *Nonlinear Function Approximation*: We evaluate SCNA-16 on 11 nonlinear functions using MSE, MAE, and MaxAE. Table IV compares general-purpose methods; Table V reports Frac-T and Frac-I, whose integer/fractional range decomposition applies only to Exp and Exp2 . Traditional baselines

TABLE IV
NONLINEAR APPROXIMATION ERRORS WITH SCNA-16. MAXAE IS MAXIMUM ABSOLUTE ERROR.

Function	MSE					MAE		MaxAE		
	Taylor	Interp	LinearLUT	NN-LUT	T-LUT	SCNA-16	NN-LUT	SCNA-16	T-LUT	SCNA-16
Exp	INF	3.60E-2	4.41E-6	1.24E-6	1.24E-6	5.02E-9	2.25E-4	1.12E-5	1.95E-3	1.08E-3
Exp2	INF	3.37E-2	2.73E-6	1.28E-6	1.24E-6	1.70E-8	2.00E-4	2.38E-5	1.95E-3	1.86E-3
Sigmoid	1.77E-3	1.25E-4	5.59E-8	2.36E-6	1.23E-6	6.78E-8	1.20E-3	2.18E-4	1.95E-3	7.20E-4
Softsign	N/A	5.24E-2	3.58E-6	3.45E-4	1.27E-6	2.11E-7	1.39E-2	2.97E-4	1.95E-3	5.70E-3
Softplus	5.98E-2	1.75E-4	7.35E-7	3.89E-6	1.26E-6	5.93E-8	1.63E-3	1.55E-4	1.95E-3	1.08E-3
Tanh	4.64E-2	4.55E-5	6.50E-8	9.68E-6	6.52E-7	1.61E-7	2.58E-3	3.34E-4	1.95E-3	1.10E-3
Arctan	N/A	1.40E-1	8.41E-6	2.25E-4	1.26E-6	3.04E-7	7.85E-3	3.67E-4	1.95E-3	3.98E-3
Erf	N/A	3.74E-4	1.85E-7	8.38E-6	4.32E-7	1.30E-7	2.30E-3	3.03E-4	1.95E-3	9.50E-4
Sin	5.23E-4	9.74E-5	1.30E-7	2.99E-3	N/A	6.47E-8	5.47E-2	2.16E-4	N/A	6.30E-4
Rsqrt	N/A	3.79E-1	8.05E-5	2.10E-2	N/A	1.42E-6	1.10E-1	7.35E-4	N/A	1.78E-2
GeLU	1.65E-1	3.27E-5	6.10E-9	6.67E-6	1.19E-6	7.84E-8	2.08E-3	1.67E-4	1.95E-3	1.06E-3
Geo. ratio	1.67E5×	2.35E4×	7.36×	356×	14.9×	1.00×	20.5×	1.00×	1.30×	1.00×

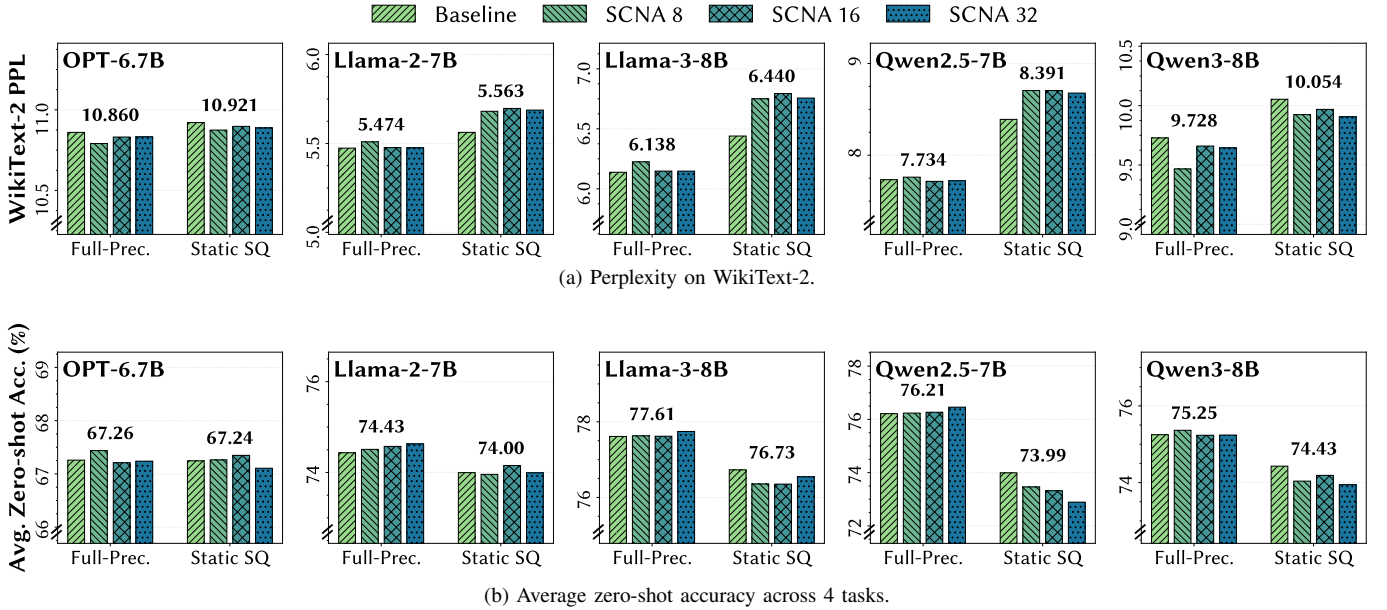


Fig. 16. SCNA model quality in full precision and static quantization. Numbers label baselines. Y-axes are truncated, with panel-specific ranges.

TABLE V
RANGE-SEPARATED EXPONENTIAL APPROXIMATION ERRORS.

Function	MSE			MAE		
	Frac-T	Frac-I	SCNA-16	Frac-T	Frac-I	SCNA-16
Exp	1.23E-8	1.07E-9	5.02E-9	2.47E-6	2.47E-6	1.12E-5
Exp2	1.75E-8	1.55E-9	1.70E-8	8.48E-6	3.57E-6	2.38E-5

use Taylor expansion and linear interpolation; NN-LUT is a learned approximator, while T-LUT uses a twofold lookup structure. T-LUT’s published absolute errors are MaxAE, not MAE. The main table retains all general-purpose MSE baselines and the learned/lookup baselines’ available MAE and MaxAE; the artifact retains the full reference data.

For each SCNA target, the input grid follows its deployment range and canonicalization recipe; Table I summarizes the principal directly parameterized functions. For symmetric or

reflected functions, the transform is applied before both approximation and error measurement. SCNA error is computed against floating-point reference functions on 1,000 uniformly spaced canonical-branch points. The GeLU entry measures the fitted $1 + \text{Erf}(x/\sqrt{2})$ branch before multiplication by $x/2$. Other methods’ entries are literature reference values and were not all rerun on this grid or at a matched parameter budget.

SCNA-16 reduces geometric-mean MSE by $356\times$ over NN-LUT [23] on 11 functions and $14.9\times$ over T-LUT [22] on nine. These aggregates use per-function baseline/SCNA ratios over finite available pairs; Taylor uses five entries. They inherit the baselines’ differing evaluation conditions. SCNA-16 reaches MSEs of 5.02×10^{-9} on Exp and 1.42×10^{-6} on Rsqrt. Frac-I has lower MSE on Exp and Exp2, whereas SCNA uses one approximation family across all 11 functions.

2) *LLM Accuracy and Quantization Robustness*: We evaluate whether SCNA maintains model quality when exact

TABLE VI
SENSITIVITY OF SCNA TO LOW-BIT QUANTIZATION.

Method	W6A6				W4A4			
	Llama2-7B		Llama3-8B		Llama2-7B		Llama3-8B	
	PPL	Acc	PPL	Acc	PPL	Acc	PPL	Acc
I-LLM	5.68	N/A	6.61	N/A	10.44	N/A	21.19	N/A
OSTQuant	5.50	74.84	6.23	77.66	5.95	71.81	7.33	74.48
w/ SCNA-8	5.54	74.80	6.32	77.62	6.00	72.20	7.44	74.08
w/ SCNA-16	5.50	74.99	6.24	77.61	5.96	72.57	7.34	74.60
w/ SCNA-32	5.50	74.75	6.24	77.62	5.95	72.39	7.35	74.16
BF16 Baseline	5.47	74.43	6.14	77.61	5.47	74.43	6.14	77.61

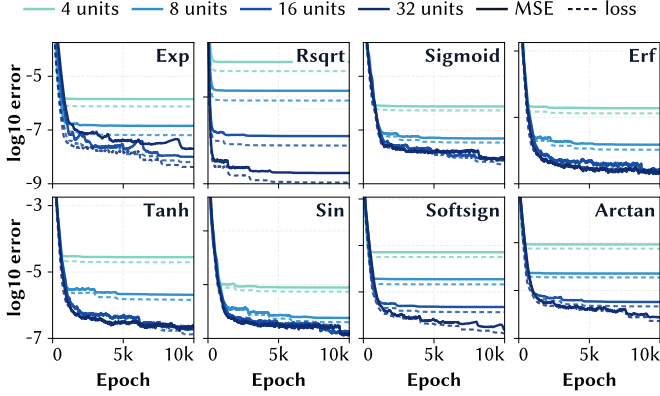


Fig. 17. Loss and MSE convergence for SCNA with 4, 8, 16, and 32 neurons.

nonlinear operators are replaced by our approximator, particularly under low-bit inference. We compare SCNA against the baseline nonlinear implementation on OPT-6.7B, Llama-2-7B, Llama-3-8B, Qwen2.5-7B, and Qwen3-8B in both BF16 and INT8. Figure 16a reports perplexity results, while Figure 16b reports mean zero-shot accuracy. SCNA- N represents that the approximator has N neurons, where $N \in \{8, 16, 32\}$. The main deployment target is SCNA-8, while SCNA-16 and SCNA-32 serve as capacity-sensitivity points. With SCNA-8, OPT-6.7B slightly improves INT8 perplexity from 10.921 to 10.875, while Llama-2-7B and Llama-3-8B increase from 5.563 to 5.681 and from 6.440 to 6.750, respectively. Results on newer models are not monotonic: Qwen2.5-7B increases from 8.391 to 8.706 under INT8, whereas Qwen3-8B improves from 10.054 to 9.924. Average zero-shot accuracy remains close to the baseline, with the largest SCNA-8 INT8 drop being only 0.5 percentage points on Qwen2.5-7B; Qwen3-8B changes from 74.4% to 74.0%. Overall, additional degradation beyond quantization is limited, though metrics vary.

3) *Accuracy with Low-Bit Dynamic Quantization*: SCNA approximation is orthogonal to the choice of quantization method and does not constrain whether the model uses static quantization, dynamic quantization, rotations, or other transformation-based quantization techniques. To further demonstrate **SCOPE**'s compatibility with advanced dynamic quantization methods, we integrate the SCNA approximator into OSTQuant [61], a quantization method using orthogonal and scaling transformations. Table VI uses the same BF16

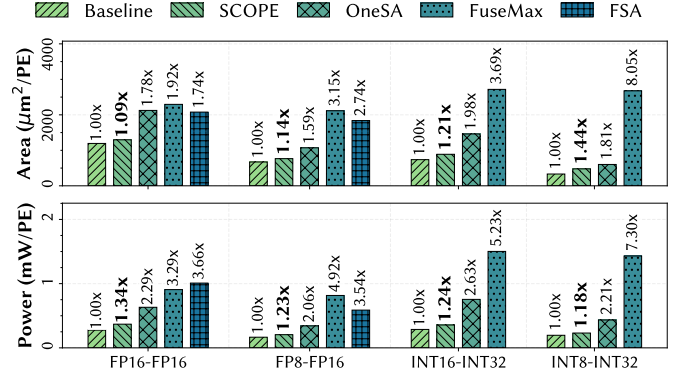


Fig. 18. Area and power consumption per PE.

baseline as Figure 16 and shows only negligible degradation from SCNA. For Llama-2-7B with SCNA-8, the W6A6 result changes from 5.50/74.84 to 5.54/74.80 in WikiText-2 perplexity/average zero-shot accuracy, and the W4A4 result changes from 5.95/71.81 to 6.00/72.20. These results confirm that SCNA approximation remains compatible with advanced quantization methods, while the proposed scale-fusion optimization provides an additional benefit for static quantization settings.

4) *SCNA Neuron-Count Scalability*: Figure 17 compares the training loss and final MSE of 4-, 8-, 16-, and 32-neuron SCNA variants on eight of the nine functions used in the neuron-count study, with Exp2 omitted. More neurons generally improve convergence and accuracy: the 32-neuron model achieves the lowest error in most cases, reducing final MSE by $97.2\times$ – $2837.8\times$ over the 4-neuron model. The 4- and 8-neuron variants often plateau at much higher error, while the 16-neuron variant closes most of the gap and often closely tracks the 32-neuron curves. The added capacity better captures fine-grained curvature and dynamic-range variation. Gains are largest for harder functions such as *Rsqrt*, *Softsign*, and *Arctan*; for smoother functions such as *Exp*, 16 neurons already capture most of the structure and can outperform 32 neurons. Thus, returns diminish near 32 neurons.

D. Hardware Overhead

We compare isolated per-PE area and power against prior array-fused nonlinear designs across four data-type/accumulation-type combinations. As shown in Figure 18, **SCOPE** consistently provides the lowest overhead among all augmented designs, requiring only $1.09\times$ – $1.44\times$ area and $1.18\times$ – $1.34\times$ power across the evaluated settings, whereas OneSA requires $1.59\times$ – $1.98\times$ area and $2.06\times$ – $2.63\times$ power, and FuseMax increases overhead much more substantially, up to $8.05\times$ area and $7.30\times$ power. This is because our array-native extension for SCNA introduces only one additional register in the left PE and one lightweight ReLU unit in the right PE of each augmented pair, avoiding heavy nonlinear structures such as comparators or LUTs.

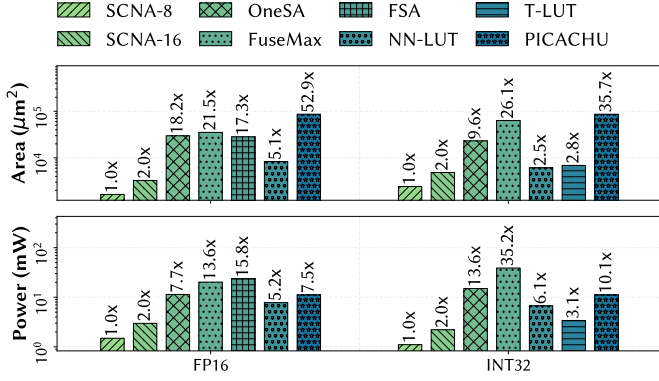


Fig. 19. Normalized incremental hardware overhead.

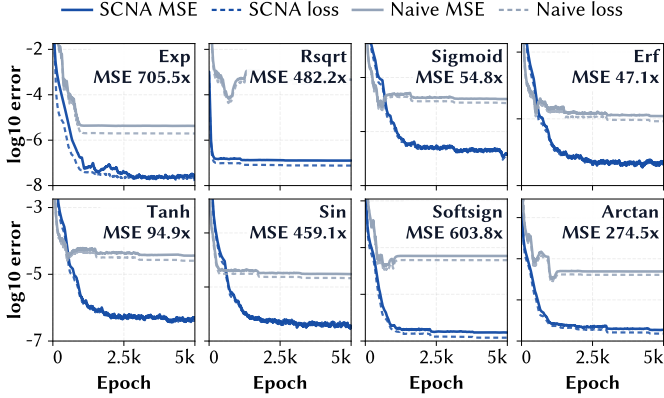


Fig. 20. Loss and MSE: 16-neuron SCNA versus unconstrained NN.

We then evaluate hardware efficiency by comparing throughput-normalized incremental area and power overhead against both array-fused designs and standalone universal approximators under FP16 and INT32. For SCNA and other array-fused designs, overhead is reported incrementally over a 32×32 baseline systolic array, then divided by nonlinear throughput. Array estimates use per-PE costs fitted from completed synthesis reports; they do not represent completed 32×32 synthesis for every design. Figure 19 compares SCNA-8 and SCNA-16, which doubles the neuron count, with competing designs. SCNA-8 uses $5.1\times$ – $52.9\times$ less area and $5.2\times$ – $15.8\times$ less power in FP16, and $2.5\times$ – $35.7\times$ less area and $3.1\times$ – $35.2\times$ less power in INT32. Across all shown baseline-precision pairs, its geometric-mean reductions are $12.8\times$ in area and $9.5\times$ in power. Even SCNA-16 remains below every competing design by avoiding large tables and complex function-specific hardware.

E. Ablation Analysis

1) *Impact of Shape Constraints*: Under the same target sampling and training budget, we compare 16-neuron SCNA with an unconstrained neural approximator on the same nine-function subset used in the neuron-count study. Figure 20 displays eight of these functions, omitting Exp^2 , and shows $47.1\times$ – $705.5\times$ lower final MSE. Gains are largest for Exp

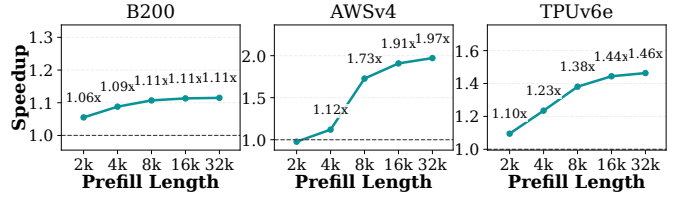


Fig. 21. Impact of scale-fused nonlinear dataflow.

($705.5\times$), Rsqrt ($482.2\times$), and Softsign ($603.8\times$), and remain significant for Sigmoid ($54.8\times$), Erf ($47.1\times$), and Tanh ($94.9\times$). The smaller gains for smooth bounded S-shaped functions reflect their easier unconstrained fitting. Shape constraints encode monotonicity, convexity, symmetry, and saturation, focusing capacity on residual fitting while regularizing optimization and avoiding the unconstrained baseline’s early plateau. This comparison evaluates the complete recipe, without isolating its constraints.

2) *Impact of Scale-Fused Dataflow*: We model type-conversion overhead using the vector throughput in Table II, with the workloads and metrics from Section VII-B. As shown in Figure 21, the scale-fused dataflow outperforms the basic dataflow except on AWSv4 at 2K ($0.98\times$), and the advantage grows with sequence length: on B200, the speedup increases from $1.06\times$ at 2K to $1.11\times$ at 32K; on TPUv6e, it rises from $1.10\times$ to $1.46\times$; and on AWSv4, it reaches $1.12\times$ at 4K, $1.73\times$ at 8K, $1.91\times$ at 16K, and $1.97\times$ at 32K. This advantage arises because scale fusion avoids extra conversion work and data movement on the critical path of the attention kernel. The benefit grows with prefill length as conversion overhead accumulates, especially on AWSv4 and TPUv6e with their lower vector throughput than B200.

VIII. CONCLUSION

We present **SCOPE**, a software-hardware co-design that makes general nonlinear evaluation an array-native capability for LLM inference. As systolic-array GEMMs accelerate, nonlinear operators increasingly bottleneck quantized inference. Prior approaches reduce individual kernel costs but struggle to combine broad operator coverage, high fidelity, and scalable hardware efficiency. **SCOPE** combines shape-constrained neural approximators (SCNA), an array-native execution mechanism on lightly extended systolic arrays, and a scale-fused dataflow for static quantization aligned with low-precision GEMMs. Across diverse nonlinear functions and five representative LLMs, **SCOPE** reduces geometric-mean approximation MSE with SCNA-16 by $356\times$ over reported NN-LUT values on 11 functions and $14.9\times$ over reported T-LUT values on nine functions, delivers up to $2.81\times$ INT8 long-context attention speedup over FlashAttention on TPUv6e, achieves up to $2.69\times$ full-prefill speedup under INT8 on B200 in the 512K-context extrapolation, and lowers throughput-normalized incremental overhead with SCNA-8 versus prior accelerators by $12.8\times$ in area and $9.5\times$ in power consumption.

A. Abstract

The SCOPE artifact contains the software models, accuracy experiments, RTL, synthesis reports, plotting scripts, and compact measurements needed to evaluate the paper. It reproduces Tables III–VI and Figures 13–21, including CPU-modeled accelerator performance, GPU model quality and nonlinear approximation accuracy, and area/power results extracted from native synthesis reports. A hardware-free reviewer path regenerates the submitted evidence and independently validates it. Fresh workflows cover CPU, GPU, RTL, and synthesis.

B. Artifact check-list (meta-information)

- **Algorithms/programs:** The artifact includes SCOPE/SCNA approximation, LLMCompass, SCALE-Sim, OSTQuant, PyTorch, Chisel RTL, and evidence validators.
- **Compilation/transformations:** The Python and Cython programs run directly after installation. The optional RTL workflow uses sbt, Scala, and Chisel to elaborate SystemVerilog.
- **Models/data:** The model set comprises OPT-6.7B, Llama-2-7B, Llama-3-8B, Qwen2.5-7B, and Qwen3-8B. The datasets are WikiText-2, Pile validation, ARC-Easy, HellaSwag, PIQA, WinoGrande, and synthetic function samples.
- **Run-time environment:** The validated environment is 64-bit GNU/Linux with Python 3.12. Fresh GPU execution additionally uses a CUDA 12.8-compatible runtime and may use Slurm.
- **Hardware:** Bundled evidence and modeled performance run on a CPU. Fresh quality experiments use NVIDIA GPUs. Repeating synthesis requires the synthesis environment described below.
- **Execution/metrics:** Automated Make workflows report latency, throughput, speedup, perplexity, zero-shot accuracy, MSE, area, and power.
- **Output:** Outputs include CSV, JSON, and Markdown validation reports, PDF and PNG plots, tables, and generated SystemVerilog.
- **Experiments:** The artifact covers Tables III–VI, Figures 13–21, RTL regeneration, and the synthesis-report audit.
- **Resources:** Bundled review requires 2 CPU cores, 8 GB RAM, 5 GB plus dependency caches, and about 15 seconds. The fresh CPU suite requires 8 cores, 16 GB RAM, 20 GB, and about 37 minutes on dual EPYC 9654. The fresh GPU suite takes about 5 hours with up to 16 H100 GPUs, excluding setup, downloads, and scheduler queue time.
- **Preparation:** Preparing the workflow takes about 10–30 minutes with network access, excluding model approval and downloads.

- **Availability:** Zenodo archive: <https://doi.org/10.5281/zenodo.22121638>. GitHub repository: <https://github.com/CLab-HKUST-GZ/SCOPE-MICRO26-AE>.

C. Description

1) *How to access:* Clone or unpack the artifact above and enter its root directory. Start with `README.md`. Experiment directories contain their own commands and expected outputs:

```
experiments/fig-*
experiments/tbl-*
```

Reproduction targets are in `expected-results`.

2) *Hardware dependencies:* Bundled evidence needs no accelerator or proprietary software. Fresh CPU experiments need roughly 8 cores and 16 GB RAM. Fresh runs for Figure 16 and Table VI need a CUDA GPU with at least 80 GB device memory. Figures 17 and 20 need at least 16 GB. Table VI additionally needs about 192 GB host memory and 200 GB free storage. Experiments may share up to 16 GPUs.

3) *Software dependencies:* Required tools are GNU Make, Bash, Python 3.10 or newer (3.12 validated), and `uv` or `venv/pip`. Fresh GPU runs use the pinned environment in:

```
requirements/accuracy.txt
```

It includes PyTorch 2.10.0+cu128, Transformers 4.51.0, and LM Evaluation Harness 0.4.4. RTL regeneration uses OpenJDK 21 and sbt. Repeating synthesis requires Synopsys Design Compiler (V-2023.12 was used) and TSMC 28 nm libraries; the synthesis target is 1 GHz. Slurm is optional.

4) *Datasets and Models:* CPU inputs and compact GPU evidence are included. Fresh GPU runs download the datasets listed above and require Hugging Face-format checkpoints prepared under `MODEL_ROOT`. Model weights and large Table VI intermediates are not redistributed. Reviewers must obtain weights under the applicable terms.

D. Installation

From the artifact root, run:

```
make setup
make evidence
make validate
```

The first command creates `.venv` and installs dependencies. An existing environment can instead be selected by setting the `PYTHON` environment variable. Network access is normally needed for initial package, model, and data fetches.

E. Experiment workflow

Fast review. Regenerate compact tables and plots in ignored run directories and validate all packaged CPU/GPU results:

```
make evidence && make validate
```

Fresh CPU/RTL. Run the modeled performance suite, hardware extraction, and Chisel elaboration:

```
make reproduce-cpu JOBS=8
```

Fresh GPU. Copy the configuration template, then set `MODEL_ROOT` and other local options:

```
cp config/local.env.example \
  config/local.env
```

Run on an allocated GPU:

```
make reproduce-gpu EXECUTOR=local \
  WORKERS=1
```

For Slurm job arrays, use:

```
make reproduce-gpu EXECUTOR=slurm \
  WORKERS=N
```

F. Evaluation and expected results

Expected checks (`status: pass`):

- 68/68 CPU checks and 36 Table III groups recomputed from 720 raw H100 samples.
- 80/80 Figure 16, 20/20 Table VI, 36/36 Figure 17, and 18/18 Figure 20 comparisons.
- All 11 SCNA rows in Table IV and zero absolute author paths.

Figures 13–15 and 21 match the reported performance and speedup targets. Figures 18–19 are regenerated from 112 filtered native synthesis report sets. Figure 16 and Table VI both use WikiText-2 perplexity and the unweighted mean accuracy over ARC-Easy, HellaSwag, PIQA, and WinoGrande. Table IV reproduces SCOPE/SCNA canonical-branch accuracy. Its other columns are literature reference values.

G. Experiment customization

Use `JOBS=N` for CPU workers and `WORKERS=N` for GPU job-array width. Make’s `-j` runs independent experiment targets concurrently. The product of concurrent targets and `WORKERS` must remain within the GPU allocation. Individual targets (for example, `make fig-16` or `make tbl-5`) permit partial reproduction. Table VI checkpoints may be reused by setting:

`TABLE5_CHECKPOINT_SOURCE`

H. Methodology

Artifact reviewing and badging follow <https://www.acm.org/publications/policies/artifact-review-and-badging-current> and <https://cTuning.org/ae>.

REFERENCES

- [1] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” 2021. [Online]. Available: <https://arxiv.org/abs/2010.11929>
- [2] Y. Bai, X. Geng, K. Mangalam, A. Bar, A. L. Yuille, T. Darrell, J. Malik, and A. A. Efros, “Sequential modeling enables scalable learning for large vision models,” in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 22 861–22 872.
- [3] X. Dong, P. Zhang, Y. Zang, Y. Cao, B. Wang, L. Ouyang, X. Wei, S. Zhang, H. Duan, M. Cao, W. Zhang, Y. Li, H. Yan, Y. Gao, X. Zhang, W. Li, J. Li, K. Chen, C. He, X. Zhang, Y. Qiao, D. Lin, and J. Wang, “InternLM-XComposer2: Mastering free-form text-image composition and comprehension in vision-language large model,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.16420>
- [4] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language models are few-shot learners,” in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, ser. NIPS ’20. Red Hook, NY, USA: Curran Associates Inc., 2020.
- [5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, ser. NIPS ’20. Red Hook, NY, USA: Curran Associates Inc., 2020.
- [6] H. T. Kung, “Why systolic architectures?” *Computer*, vol. 15, no. 1, pp. 37–46, Jan. 1982. [Online]. Available: <https://doi.org/10.1109/MC.1982.1653825>
- [7] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P.-I. Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, B. Gelb, T. V. Ghaemmaghami, R. Gottipati, W. Gulland, R. Hagmann, C. R. Ho, D. Hogberg, J. Hu, R. Hundt, D. Hurt, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khaitan, D. Killebrew, A. Koch, N. Kumar, S. Lacy, J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, K. Nix, T. Norrie, M. Omernick, N. Penukonda, A. Phelps, J. Ross, M. Ross, A. Salek, E. Samadiani, C. Severn, G. Sizikov, M. Snellman, J. Souter, D. Steinberg, A. Swing, M. Tan, G. Thorson, B. Tian, H. Toma, E. Tuttle, V. Vasudevan, R. Walter, W. Wang, E. Wilcox, and D. H. Yoon, “In-datacenter performance analysis of a tensor processing unit,” in *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 1–12.
- [8] N. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles, C. Young, X. Zhou, S. Zhou, and D. A. Patterson, “TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ser. ISCA ’23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3579371.3589350>
- [9] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “GPT3.int8(): 8-bit matrix multiplication for transformers at scale,” in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 30 318–30 332. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/c3ba4962c05c49636d4c6206a97e9c8a-Paper-Conference.pdf
- [10] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “SmoothQuant: Accurate and efficient post-training quantization for large language models,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, Eds., vol. 202. PMLR, 23–29 Jul 2023, pp. 38 087–38 099. [Online]. Available: <https://proceedings.mlr.press/v202/xiao23c.html>
- [11] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration,” in *Proceedings of Machine Learning and Systems*, P. Gibbons, G. Pekhimenko, and C. D. Sa, Eds., vol. 6, 2024, pp. 87–100. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2024/file/42a452cbafa9dd64e9ba4aa95cc1ef21-Paper-Conference.pdf
- [12] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “OPTQ: Accurate quantization for generative pre-trained transformers,” in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=tcBBPnfwxS>
- [13] B. Gao and L. Pavel, “On the properties of the softmax function with application in game theory and reinforcement learning,” 2018. [Online]. Available: <https://arxiv.org/abs/1704.00805>
- [14] D. Hendrycks and K. Gimpel, “Gaussian error linear units (GELUs),” 2023. [Online]. Available: <https://arxiv.org/abs/1606.08415>
- [15] N. Shazeer, “GLU variants improve transformer,” 2020. [Online]. Available: <https://arxiv.org/abs/2002.05202>

- [16] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," 2016. [Online]. Available: <https://arxiv.org/abs/1607.06450>
- [17] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Advances in Neural Information Processing Systems*, vol. 32. Red Hook, NY, USA: Curran Associates Inc., 2019.
- [18] T. Zadouri, M. Hoehnerbach, J. Shah, T. Liu, V. Thakkar, and T. Dao, "FlashAttention-4: Algorithm and kernel pipelining co-design for asymmetric hardware scaling," 2026. [Online]. Available: <https://arxiv.org/abs/2603.05451>
- [19] S. Ghodrati, S. Kinzer, H. Xu, R. Mahapatra, Y. Kim, B. H. Ahn, D. K. Wang, L. Karthikeyan, A. Yazdanbakhsh, J. Park, N. S. Kim, and H. Esmailzadeh, "Tandem processor: Grappling with emerging operators in neural networks," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 1165–1182. [Online]. Available: <https://doi.org/10.1145/3620665.3640365>
- [20] J. Qin, T. Xia, C. Tan, J. Zhang, and S. Q. Zhang, "PICACHU: Plug-in CGRA handling upcoming nonlinear operations in LLMs," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 845–861. [Online]. Available: <https://doi.org/10.1145/3676641.3716013>
- [21] J. Li, A. Zhurkevich, V. Agarwalla, S. Lee, and R. Anders, "Making softmax more efficient with NVIDIA Blackwell Ultra," <https://developer.nvidia.com/blog/making-softmax-more-efficient-with-nvidia-blackwell-ultra/>, 2026, NVIDIA Technical Blog, February 25, 2026.
- [22] Y. Xie, A. N. Joseph Raj, Z. Hu, S. Huang, Z. Fan, and M. Joler, "A twofold lookup table architecture for efficient approximation of activation functions," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 12, pp. 2540–2550, 2020.
- [23] J. Yu, J. Park, S. Park, M. Kim, S. Lee, D. H. Lee, and J. Choi, "NN-LUT: Neural approximation of nonlinear operations for efficient transformer inference," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, ser. DAC '22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 577–582. [Online]. Available: <https://doi.org/10.1145/3489517.3530505>
- [24] A. Cantoni, "Optimal curve fitting with piecewise linear functions," *IEEE Transactions on Computers*, vol. C-20, no. 1, pp. 59–67, 1971.
- [25] X. Dong, X. Zhu, and D. Ma, "Hardware implementation of softmax function based on piecewise LUT," in *2019 IEEE International Workshop on Future Computing (IWOFc)*, 2019, pp. 1–3.
- [26] H. C. Prashanth and M. Rao, "SOMALib: Library of exact and approximate activation functions for hardware-efficient neural network accelerators," in *2022 IEEE 40th International Conference on Computer Design (ICCD)*, 2022, pp. 746–753.
- [27] J. Lin, Y. Li, G. Chen, and T. Bourgeat, "SystolicAttention: Fusing FlashAttention within a single systolic array," 2025. [Online]. Available: <https://arxiv.org/abs/2507.11331>
- [28] N. Nayak, X. Wu, T. O. Odemuyiwa, M. Pellauer, J. S. Emer, and C. W. Fletcher, "FuseMax: Leveraging extended einsums to optimize attention accelerator design," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024, pp. 1458–1473.
- [29] R. Sun, Y. Ni, X. He, J. Zhao, and A. Zou, "ONE-SA: Enabling nonlinear operations in systolic arrays for efficient and flexible neural network inference," in *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2024, pp. 1–6.
- [30] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [31] Y. Kim, C. Denton, L. Hoang, and A. M. Rush, "Structured attention networks," in *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, Conference Track Proceedings*. OpenReview.net, 2017. [Online]. Available: <https://openreview.net/forum?id=HkE0Nvqlg>
- [32] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, "QServe: W4A8KV4 quantization and system co-design for efficient LLM serving," in *Proceedings of Machine Learning and Systems*, M. Zaharia, G. Joshi, and Y. Lin, Eds., vol. 7. MLSys, 2025. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2025/file/fbe2b2f74a2ece8070d8fb073717bda6-Paper-Conference.pdf
- [33] H. Hu, B. Xiao, S. Sun, J. Yin, Z. Zhang, X. Luo, C. Jiang, W. Xu, X. Jia, X. Liu, and M. Guo, "LiquidGEMM: Hardware-efficient W4A8 GEMM kernel for high-performance LLM serving," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 1619–1630. [Online]. Available: <https://doi.org/10.1145/3712285.3759852>
- [34] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," 2023. [Online]. Available: <https://arxiv.org/abs/2307.08691>
- [35] AWS, "Attention CTE Kernel API Reference — AWS Neuron Documentation," <https://awsdocs-neuron.readthedocs-hosted.com/en/latest/nki/library/api/attention-cte.html>, 2026, accessed: Apr. 7, 2026.
- [36] Z. Lu, H. Pu, F. Wang, Z. Hu, and L. Wang, "The expressive power of neural networks: A view from the width," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/32cbf687880eb1674a07bf17761dd3a-Paper.pdf
- [37] B. Hanin and M. Sellke, "Approximating continuous functions by ReLU nets of minimal width," 2018. [Online]. Available: <https://arxiv.org/abs/1710.11278>
- [38] S. Park, C. Yun, J. Lee, and J. Shin, "Minimum width for universal approximation," in *International Conference on Learning Representations*, 2021. [Online]. Available: <https://openreview.net/forum?id=O-XJwoIF-k>
- [39] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, no. 5, pp. 359–366, 1989. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0893608089900208>
- [40] R. Sun, S. Ye, J. Zhao, X. He, J. Lin, Y. Li, and A. Zou, "NeuralMatrix: Compute the entire neural networks with linear matrix operations for efficient inference," 2024. [Online]. Available: <https://arxiv.org/abs/2305.14405>
- [41] B. Amos, L. Xu, and J. Z. Kolter, "Input convex neural networks," in *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, ser. ICML'17. JMLR.org, 2017, pp. 146–155.
- [42] P.-J. Hoedl and G. Klambauer, "Principled weight initialisation for input-convex neural networks," in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS '23. Red Hook, NY, USA: Curran Associates Inc., 2023.
- [43] D. Runje and S. M. Shankaranarayana, "Constrained monotonic neural networks," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, Eds., vol. 202. PMLR, 23–29 Jul 2023, pp. 29338–29353. [Online]. Available: <https://proceedings.mlr.press/v202/runje23a.html>
- [44] T. Salimans and D. P. Kingma, "Weight normalization: a simple reparameterization to accelerate training of deep neural networks," in *Proceedings of the 30th International Conference on Neural Information Processing Systems*, ser. NIPS'16. Red Hook, NY, USA: Curran Associates Inc., 2016, pp. 901–909.
- [45] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, and K. Keutzer, "I-BERT: Integer-only BERT quantization," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. Meila and T. Zhang, Eds., vol. 139. PMLR, 18–24 Jul 2021, pp. 5506–5518. [Online]. Available: <https://proceedings.mlr.press/v139/kim21d.html>
- [46] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, S. Presser, and C. Leahy, "The Pile: An 800GB dataset of diverse text for language modeling," *arXiv preprint arXiv:2101.00027*, 2020.
- [47] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, "Think you have solved question answering? try ARC, the A2 reasoning challenge," 2018. [Online]. Available: <https://arxiv.org/abs/1803.05457>
- [48] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, "HellaSwag: Can a machine really finish your sentence?" in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, A. Korhonen, D. Traum, and L. Márquez, Eds. Florence, Italy:

- Association for Computational Linguistics, Jul. 2019, pp. 4791–4800. [Online]. Available: <https://aclanthology.org/P19-1472/>
- [49] Y. Bisk, R. Zellers, R. Le bras, J. Gao, and Y. Choi, “PIQA: Reasoning about physical commonsense in natural language,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 05, pp. 7432–7439, Apr. 2020. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/6239>
- [50] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi, “WinoGrande: An adversarial Winograd schema challenge at scale,” *Commun. ACM*, vol. 64, no. 9, pp. 99–106, Aug. 2021. [Online]. Available: <https://doi.org/10.1145/3474381>
- [51] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac’h, H. Li, K. McDonell, N. Muennighoff, C. Ociepa, J. Phang, L. Reynolds, H. Schoelkopf, A. Skowron, L. Sutawika, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou, “A framework for few-shot language model evaluation,” Sep. 2021. [Online]. Available: <https://doi.org/10.5281/zenodo.5371628>
- [52] H. Zhang, A. Ning, R. B. Prabhakar, and D. Wentzlaff, “LLMCompass: Enabling efficient hardware design for large language model inference,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 1080–1096.
- [53] R. Raj, S. Banerjee, N. Chandra, Z. Wan, J. Tong, A. Samajdhar, and T. Krishna, “SCALE-Sim V3: a modular cycle-accurate systolic accelerator simulator for end-to-end system analysis,” in *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2025, pp. 186–200.
- [54] NVIDIA, “NVIDIA DGX B200,” <https://www.nvidia.com/en-us/data-center/dgx-b200/>, accessed: Apr. 7, 2026.
- [55] AWS, “NeuronCore-v4 architecture,” <https://awsdocs-neuron.readthedocs-hosted.com/en/latest/about-neuron/arch/neuron-hardware/neuron-core-v4.html>, accessed: Apr. 7, 2026.
- [56] Google, “TPU v6e,” <https://docs.cloud.google.com/tpu/docs/v6e>, accessed: Apr. 7, 2026.
- [57] X. Hu, Y. Cheng, D. Yang, Z. Yuan, J. Yu, C. Xu, and S. Zhou, “I-LLM: Efficient integer-only inference for fully-quantized low-bit large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.17849>
- [58] W. Zhong, H. Feng, Z. Zhou, H. Peng, and S. Yu, “IntAttention: A fully integer attention pipeline for efficient edge inference,” in *Ninth Conference on Machine Learning and Systems*, 2026. [Online]. Available: <https://openreview.net/forum?id=CPCRITwAaP>
- [59] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avižienis, J. Wawrzyniec, and K. Asanović, “Chisel: constructing hardware in a Scala embedded language,” in *Proceedings of the 49th Annual Design Automation Conference*, ser. DAC ’12. New York, NY, USA: Association for Computing Machinery, 2012, pp. 1216–1225. [Online]. Available: <https://doi.org/10.1145/2228360.2228584>
- [60] Synopsys, Inc., “DesignWare Intellectual Property,” 2026. [Online]. Available: <https://www.synopsys.com/dw/buildingblock.php>
- [61] X. Hu, Y. Cheng, D. Yang, Z. Chen, Z. Xu, J. Yu, C. Xu, Z. Yuan, Z. Jiang, and S. Zhou, “OSTQuant: Refining large language model quantization with orthogonal and scaling transformations for better distribution fitting,” in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=rAcgDBdKnP>